

Self-Refinement Strategies for LLM-based Product Attribute Value Extraction

Alexander Brinkmann ¹ and Christian Bizer ¹

Abstract: Structured product data, in the form of attribute-value pairs, is essential for e-commerce platforms to support features such as faceted product search and attribute-based product comparison. However, vendors often provide unstructured product descriptions, making attribute value extraction necessary to ensure data consistency and usability. Large language models (LLMs) have demonstrated their potential for product attribute value extraction in few-shot scenarios. Recent research has shown that self-refinement techniques can improve the performance of LLMs on tasks such as code generation and text-to-SQL translation. For other tasks, the application of these techniques has resulted in increased costs due to processing additional tokens, without achieving any improvement in performance. This paper investigates applying two self-refinement techniques — error-based prompt rewriting and self-correction — to the product attribute value extraction task. The self-refinement techniques are evaluated across zero-shot, few-shot in-context learning, and fine-tuning scenarios using GPT-4o. The experiments show that both self-refinement techniques fail to significantly improve the extraction performance while substantially increasing processing costs. For scenarios with development data, fine-tuning yields the highest performance, while the ramp-up costs of fine-tuning are balanced out as the amount of product descriptions increases.

Keywords: Self-Refinement, Information Extraction, Large Language Models, E-Commerce

1 Introduction

Large Language Models (LLMs), such as OpenAI's GPT-4o, have been successfully applied to a wide range of tasks, including information extraction tasks such as extracting product attribute values from product descriptions [BSB25]. In order to be effective on these tasks, LLMs often rely on few-shot in-context learning and fine-tuning. Recently, methods for the automated self-refinement of prompts [Ma23] and the self-review and self-correction of model decisions [Pa24] have emerged and are successfully applied for tasks such as code generation [Ma23] and text-to-SQL translation [PR23]. At the same time, an increasing body of research [Ol23, Hu23] criticises the self-refinement approaches as for other tasks, they do not significantly improve performance while increasing the processing costs heavily due to additional tokens that need to be processed. This paper critically evaluates two self-refinement techniques for extracting attribute values from product descriptions: error-based prompt rewriting and self-correction. Error-based prompt rewriting improves the attribute definitions within prompts by analyzing errors made by the model on validation examples. Self-correction reviews and updates the initial output of an LLM if it spots wrongly

¹ University of Mannheim, Data & Web Science Group, Schloss, 68161 Mannheim, Germany,
alexander.brinkmann@uni-mannheim.de,  <https://orcid.org/0000-0002-9379-2048>;
christian.bizer@uni-mannheim.de,  <https://orcid.org/0000-0003-2367-0237>

extracted values. These two self-refinement techniques are chosen as they can be applied in a fully automated fashion and cover self-refinement during prompt engineering (error-based prompt rewriting) and post hoc self-refinement of the LLM’s output (self-correction). The self-refinement techniques are evaluated in zero-shot, few-shot in-context learning and fine-tuning scenarios. This paper makes the following contributions:

- The self-refinement techniques error-based prompt rewriting and self-correction are experimentally evaluated for the product attribute value extraction task. The self-refinement techniques are applied in zero-shot, few-shot in-context learning, and fine-tuning scenarios using GPT-4o.
- We present a detailed analysis of the impact of error-based prompt rewriting on prompt quality and the effect of self-correction on the accuracy of the extracted attribute values.

The paper is structured as follows. First, related work is reviewed. Section 3 and Section 4 describe the datasets used and the experimental setup. Section 5 introduces product attribute value extraction using few-shot learning and fine-tuning. The self-refinement strategies error-based prompt rewriting and self-correction are introduced and experimentally evaluated in Section 6 and Section 7. Code and data for replicating all experiments are available online².

2 Related Work

Attribute Value Extraction. Product attribute value extraction is a subtask of information extraction and focuses on extracting specific attribute values from unstructured text, such as product titles and descriptions [ZZH09, Ya22]. There are two variants of the task: closed-world, where a predefined schema specifies the target attributes, and open-world, where the set of target attributes is undefined and the extraction method needs to determine the attributes as well as their values [Zh22, Xu24]. We focus on closed-world product attribute value extraction. Early works use domain-specific rules to identify attribute values in product descriptions [ZZH09]. Recently, many approaches have framed product attribute value extraction as a question-answering task, using the pre-trained language model (PLM) BERT to identify the target attribute value in a product description [Ya22]. Other related works use LLMs like GPT-4 to extract attribute values from product descriptions using different prompting techniques [Fa24, BSB25].

Information Extraction using LLMs. Generative LLMs often demonstrate superior zero-shot performance compared to PLMs and exhibit higher robustness for unseen examples [Br20]. This advantage is due to extensive pre-training on large amounts of text and emergent abilities arising with large model size [We22]. LLMs have been successfully applied to information extraction tasks across various domains [Xu24]. For instance, Wang et al.[WLJ23] and Parekh et al.[Pa23] utilized OpenAI’s LLMs to extract event data from

² <https://github.com/wbgs-uni-mannheim/SelfRefinement4ExtractGPT>

unstructured text. Goel et al. [Go23] combined LLMs with human expertise to annotate patient information in medical texts.

Self-Refinement Techniques. Various techniques for model self-refinement and for correcting model outputs have been proposed recently [Pa24]. [Ma23] used automated feedback generated by an LLM to improve the readability of code. The *Self-Correction* technique evaluated in this paper also relies on such automated feedback. A critic of self-correction is that LLMs struggle to correct their responses without external feedback from tools such as code interpreters or database management systems [Hu23]. An example of how LLMs reflect on the feedback from tools to improve code successfully is given in [Sh23], while [OI23] finds GPT-4 that is not able to generate useful feedback for fixing mistakes in code. The *Error-based Prompt Rewriting* technique that is evaluated in this paper tries to improve the initial prompt using development data as a source of external feedback.

3 Datasets

This section introduces the benchmark datasets OA-Mine [Zh22] and AE-110k [Xu19] that we use for the experiments. Both datasets consist of English product offers with annotated attribute-value pairs and have been used in related work [BSB25, Ya22].

OA-Mine. We use a subset of the human-annotated product offers of the OA-Mine dataset³ [Zh22] for our experiments. The subset includes 10 product categories, with up to 80 product offers per category. Each category has between 8 to 15 attributes, resulting in a total of 115 unique attributes. Attributes with the same name in different product categories are treated as distinct attributes. We do not apply any further pre-processing to the offers.

AE-110K. The AE-110K dataset⁴ comprises triples of product titles, attributes and attribute values from the AliExpress Sports & Entertainment category [Xu19]. Product offers are derived by grouping the triples by product title. The subset includes 10 product categories, with up to 160 product offers per category. For each category, 6 to 17 attributes are known, resulting in a total of 101 unique attributes.

Development/Test Split. Table 1 contains statistics about the numbers of unique attribute-value pairs (A/V pairs), unique attribute values, and product offers for all four datasets. We use the same 60:40 development and test split based on product offers as in related work [BSB25]. The examples in the development set are labelled with ground truth attribute-value pairs. We use the development set to sample example attribute values, to select in-context demonstrations, as a validation set for error-based prompt rewriting, and as a training set for fine-tuning.

³ <https://github.com/xinyangz/OAMine/tree/main/data>

⁴ https://raw.githubusercontent.com/lanmanok/ACL19_Scaling_Up_Open_Tagging/master/publish_data.txt

Tab. 1: Descriptive statistics for OA-Mine and AE-110K.

	OA-Mine Development	Test	Total	AE-110K Development	Test	Total
A/V Pairs	3,626	2,451	6,077	2,170	1,482	3,652
Unique A/Vs	2,400	1,749	3,637	587	454	854
Product Offers	715	491	1,206	785	524	1,309

Example Extractions. Table 2 shows example product offer titles, target attributes and attribute values from the datasets. The examples (a) and (b) visualize the direct extraction meaning that the extracted attribute value is a substring of the product title. The target output is a JSON object containing all attribute-value pairs.

Tab. 2: Example product titles and attribute-value pairs from the OA-Mine and AE-110k datasets.

Dataset	OA-Mine	AE-110k
Category Attributes	Vitamin Brand, Net Content, Supplement Type, Dosage	Eyewear Sport Type, Gender Lenses Optical, Model Number
Product Title	NOW Supplements, Vitamin A (Fish Liver Oil) 25,000 IU, Essential Nutrition, 250 Softgels	Professional Men Ski Goggles Double Layers UV400 Anti-fog Big Ski Mask Skiing Snowboard Glasses
Target Output	<pre>"""json{ "Brand": "Now Supplements", "Net Content": "250", "Supplement Type": "Vitamin A", "Dosage": "25,000 IU" }"""</pre>	<pre>"""json{ "Sport Type": "Skiing", "Gender": "Men", "Lenses Optical": "UV400", "Model Number": "n/a" }"""</pre>

(a)

(b)

4 Experimental Setup

We use the LLM gpt-4o-2024-08-06⁵ for all experiments in this paper. GPT-4o is chosen because, at the time of experimenting for this paper, it is the LLM with the best performance on benchmarks measuring natural language understanding⁶. GPT-4o is accessed via the OpenAI API. GPT-4o’s temperature parameter is set to zero to reduce randomness. We report average F1-scores of three runs for each experiment and run paired t-tests to verify with 99% confidence if the F1-scores of two experiments are significantly different. The F1-score is calculated by categorizing predictions into five groups as per previous works [Xu19, Ya22, BSB25, BBB24]. The five categories are NN (no predicted value,

⁵ <https://platform.openai.com/docs/models/gp#gpt-4o>

⁶ <https://openai.com/index/hello-gpt-4o/>

no ground truth value), NV (predicted value, no ground truth value), VN (no predicted value, ground truth value), VC (predicted value exactly matches ground truth value), and VW (predicted value does not match ground truth value). The F1-score is derived from precision ($P = VC / (NV + VC + VW)$), recall ($R = VC / (VN + VC + VW)$), and the formula $F1 = 2PR / (P + R)$. Additionally, we report the average number of tokens per prompt to estimate and compare the costs of the different approaches. It is important to mention that prompt and completion tokens are summed. OpenAI charges users different prices for input tokens (2.5\$/1M tokens) and output tokens (1.25\$/1M tokens)⁷.

5 Attribute Value Extraction using In-Context Learning and Fine-Tuning

This section introduces LLM-based prompting techniques for attribute value extraction. It covers zero-shot prompts, prompts with attribute definitions generated from development data, few-shot in-context learning prompts, few-shot in-context learning prompts combined with self-consistency and fine-tuning on the development sets.

Zero-Shot. Figure 1 shows the zero-shot prompt consisting of a task description and a task input. The task description is a system chat message that describes the attribute value extraction task, lists the target attributes, defines the output as a JSON object, and explains that not available attribute values should be marked with 'n/a' in the output JSON object. The task input is a user chat message that contains the product description from which the attribute values are extracted. Examples of output JSON objects are shown in Table 2.

Task Description (System)	Extract the attribute values for 'Brand', 'Supplement Type', 'Dosage', 'Net Content' into a JSON object. Return 'n/a' if the attribute is not found.
Task Input (User)	NOW Supplements, Vitamin A (Fish Liver Oil) 25,000 IU, Essential Nutrition, 250 Softgels

Fig. 1: Zero-shot prompt.

Row *Zero-Shot* in Table 3 shows the results for zero-shot attribute value extraction experiments. The F1-scores remain below 70%, which is unsatisfying for real-world deployment. The standard deviation of the reported average F1-scores is 0.11 and 0.45 for OA-Mine and AE-110k, showing that the extracted attribute values only marginally deviate across the runs. We use the results of the zero-shot experiment as a reference point for the upcoming experiments. We report the delta between each experimental F1-score and the zero-shot F1-score as Δ ZS. We report the ratio of the number of tokens to the number of tokens used by the zero-shot prompt as token factor (TF).

Attribute Definitions. In the attribute definitions scenario, attribute definitions are appended to the task description of the zero-shot prompt. This prompt builds on the findings of

⁷ OpenAI prices as of February 2025: <https://openai.com/api/pricing/>

Tab. 3: Experimental results for attribute value extraction using in-context learning and fine-tuning.

	OA-Mine				AE-110k			
	F1	Δ ZS	Tokens	TF	F1	Δ ZS	Tokens	TF
Zero-shot	68.8	0.0	181	1.0	63.6	0.0	196	1.0
Attribute Definitions	72.2	+3.4	601	3.3	76.3	+12.7	577	2.9
Few-Shot	78.6	+9.8	1,315	7.2	83.9	+20.3	1,351	6.9
+ Self-Consistency	79.3	+10.5	3,945	21.7	84.1	+20.4	4,053	20.6
+ Attribute Definitions	79.3	+10.5	1,760	9.7	85.3	+21.7	1,727	8.8
Fine-Tuning	83.2	+14.3	172	0.9	85.1	+21.4	177	0.9

previous research demonstrating that definitions enhance the performance of LLMs for classification [Pe23] and extraction tasks [BSB25]. The example prompt in Figure 2 depicts this extension. For demonstration purposes, it contains only the definition for the attribute 'Brand'. In the experiments, a definition is appended for each attribute. Since the datasets OA-Mine and AE-110k do not contain attribute definitions, the definitions are generated by an LLM based on five attribute values. The attribute values are randomly sampled from the development set. Generating attribute definitions requires an average of 84 and 74 tokens per unique attribute for OA-Mine and AE-110k, respectively. The generated attribute definitions have an average length of 36 tokens for OA-Mine and 35 tokens for AE-110k.

Task Description (System)	Extract the attribute values for 'Brand', 'Supplement Type', 'Dosage', 'Net Content' into a JSON object. Return 'n/a' if the attribute is not found. Attribute Definitions: Brand: The 'Brand' attribute refers to the name of the company that produces the product. Examples include 'Pure Synergy', and 'Best Naturals'.
Task Input (User)	NOW Supplements, Vitamin A (Fish Liver Oil) 25,000 IU, Essential Nutrition, 250 Softgels

Fig. 2: Prompt with attribute definitions.

Table 3 shows that compared to the zero-shot results, the attribute definitions improve GPT-4o’s F1-score by 3% for OA-Mine and nearly 13% for AE-110k, which are significant differences (verified using paired t-tests with 99% confidence), but the prompts including the definitions are approximately 3 times as long as the zero-shot prompts (TF=3.3 and TF=2.9). This result underlines the usefulness of attribute definitions for extracting product attribute values.

Few-Shot In-Context Learning. In the few-shot scenario, demonstrations with annotated attribute-value pairs are presumably available as a development set. For few-shot learning, the zero-shot prompt and the prompt with attribute definitions are extended with demonstrations. Figure 3 shows the extension of the zero-shot prompt. Each demonstration consists of a

demonstration task input and a demonstration task output. The demonstration task input is a user message containing a product description. The demonstration task output is an assistant message with the extracted attribute-value pairs formatted as a JSON object. We use demonstrations that are semantically similar to the current product offer. For selecting these demonstrations, the demonstrations of the development set are embedded using OpenAI’s embedding model `text-embedding-ada-002`⁸. The embedded demonstrations with the greatest cosine similarity to the embedded task input are considered to be semantically similar. Following [BSB25], we add 10 demonstrations to each prompt.

<i>Task Description (System)</i>	Extract the attribute values for 'Brand', 'Supplement Type', 'Dosage', 'Net Content' into a JSON object. Return 'n/a' if the attribute is not found.
<i>Demonstration – Task Input (User)</i>	NOW Supplements, Vitamin C Crystals (Ascorbic Acid), Antioxidant Protection*, 1-Pound
<i>Demonstration – Task Output (Assistant)</i>	```json { "Brand": "NOW Supplements", "Supplement type": "Ascorbic Acid", "Dosage": "n/a", "Net content": "1-Pound" } ```
<i>Task Input (User)</i>	NOW Supplements, Vitamin A (Fish Liver Oil) 25,000 IU, Essential Nutrition, 250 Softgels

Fig. 3: Few-shot prompt.

Row *Few-Shot* in Table 3 reports the results of the few-shot experiments. As expected, few-shot in-context learning with semantically similar demonstrations significantly improves GPT-4o’s F1-score but more than doubles the number of tokens per prompt compared to adding attribute definitions (e.g. the TF for OA-Mine rises from 3.3 to 7.2). The gains show that GPT-4o requires demonstrations to achieve F1-scores above or at least close to 80%. Attribute definitions increase the F1-score by 1%, which is a significant increase according to the t-tests, but adds around 400 tokens to the prompt (TF=9.7 and TF=8.8).

Few-Shot Self-Consistency. Self-consistency extends the few-shot in-context learning prompt by sampling three outputs from the LLM and selecting the most consistent answer through majority voting for each attribute. Each run attribute order in the prompt is shuffled to avoid an ordering bias [ZYW25]. Related work proposes self-consistency as an alternative to self-refinement with similar token usage [Hu23]. Row *Self-consistency* in Table 3 reports the experimental results. Due to the three runs, few-shot in-context learning with self-consistency costs three times the number of tokens compared to few-shot in-context learning. The ensemble runs have a low deviation in the extracted attribute values explaining the insignificant performance changes.

Fine-Tuning. In-context learning provides task-specific knowledge to the LLM via the prompt. Fine-tuning uses training data to encode task-specific knowledge into the parameters of the LLM. At runtime, this task-specific knowledge is implicitly used to extract attribute

⁸ <https://platform.openai.com/docs/guides/embeddings/>

values. In preparation for fine-tuning, the development records are formatted with the zero-shot prompt consisting of a task description, an input and an output containing attribute-value pairs. To execute the fine-tuning, the pre-processed datasets are uploaded to OpenAI's fine-tuning API⁹ and GPT-4o is trained for three epochs on the uploaded datasets using OpenAI's default parameters. Row *Fine-Tuning* in Table 3 reports the fine-tuning results for GPT-4o. The fine-tuned GPT-4o LLMs achieve the highest average F1-score with the lowest token usage during application (TF=0.9). But especially on the dataset AE-110k, few-shot in-context learning is a competitive alternative to fine-tuning. The initial investment required for fine-tuning is 394k and 431k tokens, respectively, for OA-Mine and AE-110k. Based on the number of tokens and the token cost per product offer, a break-even point concerning the number of test product offers for using fine-tuning GPT-4o instead of few-shot in-context learning can be calculated as follows: $FineTuningCost / (TokenCost_{FewShot} - TokenCost_{FineTuning})$. Using this formula the average break-even point for the datasets is 6,666 product offers. For scenarios in which attribute values have to be extracted from more than 6,666 product offers, it is cheaper to invest in fine-tuning than to use the base model with the longer in-context learning prompts.

6 Error-based Prompt Rewriting

This section introduces and evaluates the self-refinement technique of error-based prompt rewriting. The method is combined with the zero-shot and few-shot in-context learning prompts that use attribute definitions introduced in Section 5.

Prompting Technique. Error-based prompt rewriting uses the development set to improve the attribute definitions in the prompts. It assumes that better attribute definitions improve the product attribute value extraction [Pe23]. For error-based prompt rewriting, the prompts with attribute definitions are run on five randomly selected product offers from the development set for each category to extract attribute values. The extracted attribute values are compared to the ground truth from the development set to identify incorrectly extracted values. For each attribute with extraction errors, the prompt shown in Figure 4 is populated with the existing attribute definition and a list of product descriptions with incorrectly and correctly extracted attribute values. In the depicted example, '60' instead of '60 Capsules' is expected. The LLM responds to the prompt with a rewritten attribute definition. Table 4 illustrates how the attribute definition for the Net Content is rewritten to be more specific. Error-based prompt rewriting is repeated three times to evaluate the product attribute value extraction on up to 15 product descriptions to improve the attribute definitions.

Discussion of Results. Rewriting attribute definitions increases token usage. In the zero-shot scenario, 270k tokens are required on OA-Mine for rewriting attribute definitions, while 350k tokens are consumed on AE-110k. These values rise to 378k and 389k tokens in the few-shot scenario. Table 5 reports the experimental results of the product attribute

⁹ <https://platform.openai.com/docs/guides/fine-tuning>

Task Description (System)	You are provided with an attribute definition that is used to extract attribute values from product descriptions and a list of product descriptions with incorrectly and correctly extract attribute values. Improve the attribute definition based on the incorrect and correct attribute values. Attribute values should be mentioned in "" and should naturally occur in the attribute definition. Respond with JSON object in the format: attribute: attribute definition with example values.
Task Input (User)	Type: The 'Net content' attribute indicates the amount of product contained in the package, which can be expressed in various units like '4 oz', '500', or '1 lb (454 Grams)'. Product Offer: Thorne Research - Quercenase - Quercetin Phytosome Supplement with Bromelain - 60 Capsules Incorrect value: 60 Capsules Correct value: 60

Fig. 4: Prompt for error-based rewriting of attribute definitions.

Tab. 4: Definitions for the attribute 'Net Content' of a vitamin product.

Extraction Error	Attribute Definition	Rewritten Attribute Definition
Extracted Value: "60 Capsules"	The 'Net Content' attribute indicates the amount of product contained in the package, which can be expressed in various units like '4 oz', '500', or '1 lb (454 Grams)'.	The 'Net Content' attribute specifies the quantity of product within a package. For products like capsules, or softgels, it is expressed as a numerical value, such as '60'.
Expected Value: "60"		For liquid products, it is expressed in fluid ounces, such as '0.08 Fl Oz'. For products sold by weight, it should be expressed in pounds or grams, such as '1 lb (454 Grams)'.

extraction without and with rewritten attribute definitions. In the zero-shot scenario, the rewritten definitions insignificantly degrade GPT-4o’s performance (verified by t-tests with 99% confidence) and cause an increased token usage (e.g. the TF for OA-Mine rises from 3.3 to 8.6). In the few-shot learning configuration, the performance does not significantly change, though token usage rises again (e.g. the TF for OA-Mine rises from 9.7 to 13.9).

Analysis of Rewritten Attribute Definitions. The analysis of error-based attribute definition rewriting focuses on the OA-Mine dataset in the zero-shot scenario. To reduce the manual effort of analyzing all changes, GPT-4o is used to analyze the attribute definition. GPT-4o analyzes the length of the attribute definitions, the level of detail of the original and the rewritten attribute definition, counts the number of example values per attribute definition and checks if values are excluded in an attribute definition. We manually review 10 analyzed rewriting operations to estimate how well GPT-4o performs this analysis. In the manual assessment, the level of detail and the mention of excluded example values are always correct. The example value counts are in 80% of the cases correct. With a tolerance range of

Tab. 5: Experimental results for error-based prompt rewriting.

	OA-Mine				AE-110k			
	F1	Δ ZS	Tokens	TF	F1	Δ ZS	Tokens	TF
Attribute Definitions	72.2	+3.4	601	3.3	76.3	+12.7	577	2.9
+ Rewriting	71.8	+2.9	1,569	8.6	74.4	+10.8	1,872	9.5
Few Shot + Attr. Defs.	79.3	+10.5	1,760	9.7	85.3	+21.7	1,727	8.8
+ Rewriting	79.0	+10.2	2,520	13.9	85.5	+21.8	2,650	13.5

two, all example value counts are valid. Hence, GPT-4o is useful for this analysis. Over three iterations, GPT-4o completed 176 out of 345 possible rewrites. The 345 possible rewrites are calculated by multiplying the number of unique attributes (115) in OA-mine by three because every iteration each unique attribute definition can but must not be rewritten once. The quantitative analysis of the attribute definitions shows that 97% of the rewritten attribute definitions are longer than the original attribute definition. The longer attribute definitions explain the higher TF reported in Table 5. 93% of GPT-4o’s rewriting operations enhance the level of detail. In 65% of the operations, the level of detail is increased by adding example values and in 31% of the operations, the level of detail is increased by explicitly excluding attribute values. The enhanced level of detail through additional information and the inclusion and exclusion of attribute values leads to an overfitting of the attribute definitions to the development set, which harms the LLM’s performance on the test set.

7 Self-Correction

This section introduces and evaluates the post hoc self-refinement technique self-correction. Self-correction can be combined with all prompts introduced in Section 5.

Prompting Technique. Motivated by related work [Ma23], the LLM post hoc reviews and updates its initially extracted attribute values. Therefore, a first prompt instructs the LLM to extract attribute-value pairs from the input. This first prompt is one of the prompts introduced in Section 5 depending on the scenario (zero-shot, few-shot in-context learning or fine-tuning). The output of the first prompt is sent to the same LLM again with a request to reflect on and correct erroneously extracted attribute values. Figure 5 illustrates the second self-correction prompt. A criticism of related work on the original self-refinement paper is that the refinement prompt provides additional task-related information [Hu23]. To ensure that similar information is provided by the initial prompt and the self-correction prompt, attribute definitions and few-shot in-context learning demonstrations are added to the prompt if the initial prompt contains them. In the fine-tuning scenario, the fine-tuned GPT-4o executes the self-correction prompt.

Task Description (System)	Check if the attribute values for the attributes 'Brand', 'Supplement Type', 'Dosage', 'Net Content' are correctly extracted. Respond with a JSON object that contains correctly extracted attribute values. Return 'n/a' if the attribute is not found.
Task Input (User)	NOW Supplements, Vitamin A (Fish Liver Oil) 25,000 IU, Essential Nutrition, 250 Softgels Extracted Attribute Values: { "Brand": "NOW Supplements", "Supplement type": "Vitamin A", "Dosage": "25,000 IU", "Net content": "n/a" }

Fig. 5: Prompt for self-correction.

Discussion of Results. Table 6 shows that in the zero-shot scenario, the performance gains from self-correction are insignificant (verified by t-tests with 99% confidence) and do not justify the substantial increase in computational cost (TF=2.6). In the few-shot scenario with in-context learning demonstrations, self-correction does not significantly alter GPT-4o’s performance while more than doubling token usage (e.g. the TF for OA-Mine rises from 7.2 to 15.2). In the fine-tuning scenario, the performance differences are insignificant but again the token usage doubles (e.g. the TF for OA-Mine rises from 0.9 to 2.5). These findings indicate that self-correction is not cost-effective and only marginally influences the performance of GPT-4o for attribute value extraction across different learning scenarios.

Tab. 6: Experimental results for self-correction.

	OA-Mine				AE-110k			
	F1	Δ ZS	Tokens	TF	F1	Δ ZS	Tokens	TF
Zero-shot	68.8	0.0	181	1.0	63.6	0.0	196	1.0
+ Self-Correction	69.3	+0.4	467	2.6	63.8	+0.1	503	2.6
Attribute Definitions	72.2	+3.4	601	3.3	76.3	+12.7	577	2.9
+ Self-Correction	72.5	+3.6	1,305	7.2	77.2	+13.5	1,285	6.5
Few-Shot	78.6	+9.8	1,315	7.2	83.9	+20.3	1,351	6.9
+ Self-Correction	78.5	+9.6	2,751	15.2	83.7	+20.0	2,815	14.3
+ Attribute Definitions	79.3	+10.5	1,760	9.7	85.3	+21.7	1,727	8.8
+ Attr. Def. & Self-Corr.	78.7	+9.8	3,589	19.8	84.9	+21.3	3,585	18.3
Fine-Tuning	83.2	+14.3	172	0.9	85.1	+21.4	177	0.9
+ Self-Correction	82.9	+14.1	449	2.5	85.2	+21.6	457	2.3

Analysis of Self-Corrected Attribute Values. We analyze the self-correction in detail by distinguishing three possible outcomes of the self-correction step: (1) a wrong value is corrected (Improvement), (2) a previously correct value is corrupted (Corruption), and (3) the update of the extracted value does not correct a wrong value but just changes it (Still wrong). Table 7 provides examples of the three outcomes. The target attribute value is underlined in the product description. A quantitative analysis of the zero-shot scenario

shows that 64% of the 165 attribute value updates on OA-Mine and 90% of the 423 attribute value updates have no impact because the extracted value and the updated value are incorrect. Across the zero-shot, few-shot in-context learning and fine-tuning scenarios, corruptions happen more often than improvements explaining the marginal decrease in performance when self-correction is applied. For instance, in the zero-shot scenario on OA-Mine 19% of the corrections are improvements while 16% of the corrections are corruptions. On both datasets, the amount of corrected values decreases from zero-shot to fine-tuning. For example on OA-Mine, 165 values are updated in the zero-shot scenario while 106 values are updated after fine-tuning. The LLM seems to correct attribute values where it is undecided between two possible values. These undecided attribute values change with in-context learning demonstrations. The updated extracted attribute values of the zero-shot and the few-shot in-context learning scenario overlap only by 13 and 25 values for OA-Mine and AE-110k, respectively.

Tab. 7: Possible outcomes of self-correction.

Outcome	Attribute	Product Description	Extracted Value	Corrected Value
Improvement	Net Content	Nature’s Path Organic Oatmeal, (Pack of 6, <u>11.3 Oz Boxes</u>)	11.3 Oz Boxes ✗	11.3 Oz ✓
Corruption	Pack size	Goddess Garden - SPF 50 Sunscreen Stick - <u>1 Unit</u>	1 Unit ✓	n/a ✗
Still wrong	Item form	Good Natured Lavender Laundry Soda/Detergent <u>52 load bag 32 oz.</u>	Soda/Detergent ✗	Laundry Soda ✗

8 Conclusion

This paper evaluated self-refinement strategies for large language models (LLMs) in the context of product attribute value extraction. The experimental evaluation examined two self-refinement techniques: error-based prompt rewriting and self-correction, across zero-shot, few-shot in-context learning, and fine-tuning scenarios using GPT-4o. Error-based prompt rewriting and self-correction increased computational cost due to higher token consumption without significant gains in extraction performance. While error-based prompt rewriting improved the level of detail of attribute definitions by adding and excluding example values, this likely led to overfitting to the development set. Self-correction occasionally corrected wrongly extracted attribute values but also introduced new errors, leading to insignificant changes in the extraction performance. Overall, fine-tuning without self-refinement achieved the highest F1-score and is the most cost-efficient approach for scenarios where attribute values need to be extracted from a large number of product descriptions.

Bibliography

- [BBB24] Brinkmann, Alexander; Baumann, Nick; Bizer, Christian: Using LLMs for the Extraction and Normalization of Product Attribute Values. In: *Advances in Databases and Information Systems*. pp. 217–230, 2024.
- [Br20] Brown, Tom; Mann, Benjamin; Ryder, Nick et al.: Language Models are Few-Shot Learners. In: *Advances in Neural Information Processing Systems*. volume 33, pp. 1877–1901, 2020.
- [BSB25] Brinkmann, Alexander; Shraga, Roei; Bizer, Christian: ExtractGPT: Exploring the Potential of Large Language Models for Product Attribute Value Extraction. In: *Information Integration and Web Intelligence*. pp. 38–52, 2025.
- [Fa24] Fang, Chenhao; Li, Xiaohan; Fan, Zezhong et al.: LLM-Ensemble: Optimal Large Language Model Ensemble Method for E-commerce Product Attribute Value Extraction. In: *Proceedings of the 47th ACM SIGIR International Conference on Research and Development in Information Retrieval*. pp. 2910–2914, 2024.
- [Go23] Goel, Akshay; Gueta, Almog; Gilon, Omry et al.: LLMs Accelerate Annotation for Medical Information Extraction. In: *Proceedings of the 3rd Machine Learning for Health Symposium*. pp. 82–100, 2023.
- [Hu23] Huang, Jie; Chen, Xinyun; Mishra, Swaroop et al.: Large Language Models Cannot Self-Correct Reasoning Yet. In: *Proceedings of the Twelfth International Conference on Learning Representations*. 2023.
- [Ma23] Madaan, Aman; Tandon, Niket; Gupta, Prakhar; Clark: Self-Refine: Iterative Refinement with Self-Feedback. *Advances in Neural Information Processing Systems*, 36:46534–46594, 2023.
- [OI23] Olausson, Theo X.; Inala, Jeevana Priya; Wang, Chenglong et al.: Is Self-Repair a Silver Bullet for Code Generation? In: *Proceedings of the Twelfth International Conference on Learning Representations*. 2023.
- [Pa23] Parekh, Tanmay; Hsu, I-Hung; Huang, Kuan-Hao et al.: GENEVA: Benchmarking Generalizability for Event Argument Extraction with Hundreds of Event Types and Argument Roles. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. pp. 3664–3686, 2023.
- [Pa24] Pan, Liangming; Saxon, Michael; Xu, Wenda et al.: Automatically Correcting Large Language Models: *Surveying the Landscape of Diverse Automated Correction Strategies*. *Transactions of the Association for Computational Linguistics*, 12:484–506, 2024.
- [Pe23] Peskine, Youri; Korenčić, Damir; Grubisic, Ivan et al.: Definitions Matter: Guiding GPT for Multi-label Classification. In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. pp. 4054–4063, 2023.
- [PR23] Pourreza, Mohammadreza; Rafiei, Davood: DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. *Advances in Neural Information Processing Systems*, 36:36339–36348, 2023.
- [Sh23] Shinn, Noah; Cassano, Federico; Gopinath, Ashwin et al.: Reflexion: language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023.

- [We22] Wei, Jason; Tay, Yi; Bommasani, Rishi et al.: Emergent Abilities of Large Language Models. *Transactions on Machine Learning Research*, 2022.
- [WLJ23] Wang, Xingyao; Li, Sha; Ji, Heng: Code4Struct: Code Generation for Few-Shot Event Structure Prediction. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. volume 1, pp. 3640–3663, 2023.
- [Xu19] Xu, Huimin; Wang, Wenting; Mao, Xin et al.: Scaling up Open Tagging from Tens to Thousands: Comprehension Empowered Attribute Value Extraction from Product Title. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. pp. 5214–5223, 2019.
- [Xu24] Xu, Derong; Chen, Wei; Peng, Wenjun et al.: Large language models for generative information extraction: a survey. *Frontiers of Computer Science*, 18(6):186357, 2024.
- [Ya22] Yang, Li; Wang, Qifan; Yu, Zac et al.: MAVE: A Product Dataset for Multi-source Attribute Value Extraction. In: *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*. pp. 1256–1265, 2022.
- [Zh22] Zhang, Xinyang; Zhang, Chenwei; Li, Xian et al.: OA-Mine: Open-World Attribute Mining for E-Commerce Products with Weak Supervision. In: *Proceedings of the ACM Web Conference 2022*. pp. 3153–3161, 2022.
- [ZYW25] Zhang, Zikang; You, Wangjie; Wu, Tianci: A Survey of Generative Information Extraction. In: *Proceedings of the 31st International Conference on Computational Linguistics*. pp. 4840–4870, 2025.
- [ZZH09] Zhang, Liyi; Zhu, Mingzhu; Huang, Wei: A Framework for an Ontology-based E-commerce Product Information Retrieval System. *Journal of Computers*, 4(6):436–443, 2009.